

Le rendu doit être dactylographié et au format PDF, les autres formats ne seront pas corrigés. Pour les questions commençant par “*Présenter une solution*”, vous devez expliquer textuellement comment fonctionne votre solution. Pour les questions commençant par “*Donner un algorithme*”, vous devez donner l’algorithme, avec l’explication de son fonctionnement et des commentaires.

Pour les algorithmes, vous ne disposez pas d’autres objets/fonctions que des listes, files ou piles et leurs fonctions associées définies dans le cours.

**Attention, une réponse ne sera pas notée si:**

- Le soin apporté à la rédaction et à la copie sont insuffisants,
- Toutes les réponses ne sont pas justifiées par un raisonnement clair et précis ou expliquées.

### **Exercice 1 : Balles rebondissantes** (10 points)

Nous disposons de balles rebondissantes qui, lancées contre un mur, rebondissent en direction de la personne qui lance. En fonction de sa distance au mur  $p$ , en nombre de pas, la personne peut rattraper la balle ou non. En dessous d’un nombre de pas  $p_{\max}$ , la balle rebondit suffisamment pour pouvoir être rattrapée et réutilisée. Au delà de  $p_{\max}$  la balle ne rebondit plus suffisamment pour pouvoir être rattrapée, elle est perdue. La valeur de  $p_{\max}$  est la même pour toutes les balles et la taille de la pièce est limitée à  $P$  pas, donc  $0 \leq p_{\max} \leq P$ . Notre objectif est de trouver la valeur de  $p_{\max}$  en un nombre minimal d’essais  $e$ .

**Question 1.1 :** *Proposer une stratégie permettant de trouver  $p_{\max}$  avec une balle. Justifiez votre réponse.*

**Question 1.2 :** *Quel est le nombre minimum d’essais nécessaires pour être sûr de trouver  $p_{\max}$  dans une pièce de taille  $P$  avec une balle?*

Attention, nous devons être sûr de trouver  $p_{\max}$ . Nous ne pouvons donc pas faire un lancé qui ne permettrait pas de recevoir la balle s’il ne permet pas d’en déduire  $p_{\max}$  puisque après nous n’aurions plus de balle pour tester d’autres valeurs de  $p$ .

**Question 1.3 :** *Proposer une stratégie permettant de trouver  $p_{\max}$  avec deux balles lorsque la taille de la pièce est  $P = 10$ . Quel est le nombre minimal d’essais  $e$  nécessaires pour trouver à coup sûr  $p_{\max}$ ?*

Nous nous intéressons maintenant au problème général, avec un nombre de balles  $n$  et une taille de pièce  $P$ . Dans ce cas, trouver directement  $p_{\max}$  est difficile mais il est possible, pour un nombre d’essais  $e$  et un nombre de balles  $n$ , de trouver la taille maximale de la pièce  $P$  dans laquelle nous pouvons trouver  $p_{\max}$ . Pour cela nous partons d’une distance  $d$  pour faire le premier essai. Si la personne rattrape la balle cela signifie que la distance  $d$  est telle que  $d \leq p_{\max} \leq P$ . Il reste alors  $n$  balles mais plus que  $e - 1$  essais possibles.

Si la personne ne rattrape pas la balle cela signifie que  $p_{\max} < d$  et il reste alors  $n - 1$  balles mais plus que  $e - 1$  essais possibles.

**Question 1.4 :** Donner la relation de récurrence qui permet de trouver la taille maximale  $P$  de la pièce dans laquelle il est possible de trouver  $p_{\max}$ .

**Question 1.5 :** Donner un algorithme de programmation dynamique retournant la taille maximale  $P$  de la pièce pour  $n$  balles et  $e$  essais.

**Question 1.6 :** En déduire une stratégie permettant de trouver à coup sûr la valeur de  $p_{\max}$  dans une pièce de taille  $P$ , avec  $n$  balles.

### **Correction Exercice 1 : Balles rebondissantes**

Ce problème est un problème classique d'optimisation qui est référencé dans la littérature comme étant le "egg dropping problem"

À noter que l'énoncé n'est pas suffisamment clair sur le fait que la balle rebondit ou ne rebondit pas pour la valeur de  $p_{\max}$ . Ici nous considérons que  $p_{\max}$  est la plus grande distance à laquelle la balle rebondit suffisamment pour être rattrapée.

**Solution question 1.1 :** Proposer une stratégie permettant de trouver  $p_{\max}$  avec une balle. Justifiez votre réponse.

Il n'y a pas d'autre stratégie possible que de tester avec chaque nombre de pas, de 1 à  $p_{\max}$ . Sinon, si nous ne testons pas chacun des nombres de pas et que nous ne rattrapons pas la balle, alors la valeur de  $p_{\max}$  peut être soit celle de notre dernier lancé, soit toute valeur entre ce dernier lancé et la lancé courant. Mais nous n'avons plus de balle pour le tester.

**Solution question 1.2 :** Quel est le nombre minimum d'essais nécessaires pour être sûr de trouver  $p_{\max}$  dans une pièce de taille  $P$  avec une balle?

Le nombre d'essai minimal est donc de  $P$ . En effet, si  $p_{\max} = P$  il faut tester jusqu'à  $P$ .

**Solution question 1.3 :** Proposer une stratégie permettant de trouver  $p_{\max}$  avec deux balles lorsque la taille de la pièce est  $P = 10$ . Quel est le nombre minimal d'essais  $e$  nécessaires pour trouver à coup sûr  $p_{\max}$ ?

Il est tentant de faire cette recherche par dichotomie, c'est à dire de commencer à 5 pas et ensuite de diviser la distance par deux, mais la dichotomie ne prend pas en compte le nombre de balles. Ainsi, si nous lançons la balle depuis 5 pas et quelle ne revient pas, nous n'avons plus qu'une balle et nous devons donc tester successivement les valeurs de 1 à 4. Au pire cas la valeur de  $p_{\max}$  est 4 et nous devons donc faire les 4 essais, ce qui fait 5 essais en tout avec le test à 5 pas.

La recherche par dichotomie permet donc de réduire le nombre de tests par rapport à la technique utilisée pour la question 1 mais elle n'est pas optimale car, une fois la première balle perdue, la dichotomie ne peut plus être appliquée et nous sommes obligés de revenir à la technique de la question 1. Nous pouvons également noter que l'exploration des intervalles n'est pas équivalente. Ainsi, si la balle n'a pas été rattrapée en 5, alors nous n'avons plus qu'une balle pour explorer de 0 à 4, alors que, si la balle est rattrapée, nous

avons deux balles pour explorer de 6 à 10. Diminuer la taille de l'intervalle du dessous permet alors de trouver une meilleure solution.

Dans le cas qui nous intéresse, pour avoir le nombre minimal d'essais, il faut commencer à 4 pas. Si nous ne rattrapons pas la balle, nous savons que  $p_{\max}$  est inférieur ou égal à 3 et il nous reste une balle pour tester les distances de 1 à 3, soit un total de 4 essais. Si nous rattrapons la balle nous savons que  $4 \leq p_{\max} \leq 10$ . Pour notre second essai nous relançons la balle depuis 7 pas ( $4+(4-1)$ ). Si nous ne rattrapons pas la balle, il nous faut 2 essais pour tester les valeurs de 5 et 6 pas avec la balle restante soit 4 essais au total. Si nous rattrapons la balle nous reculons à 9 pas ( $4+(4-1)+(4-2)$ ). Si nous ne rattrapons pas la balle, il nous en reste une pour lancer à 8 pas et nous aurons fait 4 essais. Si nous la rattrapons, nous pouvons tester à 10 pas avec encore une fois 4 essais.

Avec cette stratégie nous trouvons  $p_{\max}$  en maximum 4 essais. Cependant, pour savoir où commencer, il est nécessaire de connaître le nombre maximum d'essais nécessaires. Or trouver cette valeur directement n'est pas simple. Nous voyons dans la question suivante comment, calculer la taille maximale de la pièce pour laquelle nous pouvons trouver  $p_{\max}$  à coup sûr avec un nombre  $n$  de balles et un nombre  $e$  d'essais fixés.

**Solution question 1.4 :** *Donner la relation de récurrence qui permet de trouver la taille maximale  $P$  de la pièce dans laquelle il est possible de trouver  $p_{\max}$ .*

Nous appelons  $T(n, e)$  la fonction qui permet de calculer la taille maximale de la pièce pour laquelle nous pouvons trouver  $p_{\max}$  à coup sûr avec  $n$  balles et  $e$  essais. La fonction  $T$  est définie par:

$$\begin{aligned} T(1, e) &= e \\ T(n, e) &= T(n-1, e-1) + 1 + T(n, e-1) \text{ pour } n > 1 \end{aligned} \tag{1}$$

**Solution question 1.5 :** *Donner un algorithme de programmation dynamique retournant la taille maximale  $P$  de la pièce pour  $n$  balles et  $e$  essais.*

L'algorithme 1 permet de calculer, en utilisant la formule précédente, la taille maximale de la pièce pour laquelle  $p_{\max}$  peut être trouvé à coup sûr.

---

**Algorithme 1 :** Algorithme de calcul de la taille de pièce maximale permettant de trouver  $p_{\max}$  avec  $e$  essais et  $n$  balles.

---

1 Fonctions **taillePieceMax(n,e)**

**Entrées :**

$n$ : nombre de balles

$e$ : nombre d'essais

**Données :**

$TaillePiece[n,e]$ : tableau de tailles maximales

$i$ : entier, indice des balles

$j$ : entier, indice des essais

2 **début**

3   **pour**  $i$  de 0 à  $n$  **faire**  $TaillePiece[i,0] \leftarrow 0$

4   **pour**  $j$  de 0 à  $e$  **faire**  $TaillePiece[1,j] \leftarrow 1$

5   **pour**  $i$  de 1 à  $n$  **faire**

6     **pour**  $j$  de 2 à  $e$  **faire**

7        $TaillePiece[i,j] \leftarrow TaillePiece[i-1,j-1] + 1 + TaillePiece[i,j-1]$

---

**Solution question 1.6 :** En déduire une stratégie permettant de trouver à coup sûr la valeur de  $p_{\max}$  dans une pièce de taille  $P$ , avec  $n$  balles.

Comme il faut que les nombres  $e$  d'essais et  $n$  de balles permettent de trouver à coup sûr la valeur de  $p_{\max}$  pour notre pièce, il suffit de rechercher, dans le tableau  $TaillePiece$ , le nombre d'essais minimums  $e$  nécessaires pour trouver  $p_{\max}$  à coup sûr, donc une valeur supérieure ou égale à la taille de notre pièce. Nous avons ainsi les valeurs de  $n$ ,  $P$  et  $e$ . Nous devons maintenant trouver où commencer.

Si nous faisons un premier essai à une certaine distance, et que nous ne rattrapons pas la balle, alors il ne nous reste plus que  $n - 1$  balles et  $e - 1$  essais. La taille maximale de la pièce qui peut être testée avec  $n - 1$  balles et  $e - 1$  essais est donnée par notre fonction  $T(n - 1, e - 1)$ . En commençant à la distance  $T(n - 1, e - 1) + 1$ , avec  $n$  balles nous sommes sûrs, si le premier essai est infructueux de pouvoir tester les distances inférieures. De même, puisque la valeur de  $e$  a été choisie en fonction de  $P$ , nous sommes sûrs de pouvoir tester les valeurs supérieures.

Si, après notre lancé en  $T(n - 1, e - 1) + 1$ , nous en rattrapons pas la balle, il faut recommencer en  $T(n - 2, e - 2) + 1$ . À l'inverse, si nous rattrapons la balle nous pouvons recommencer en  $T(n - 1, e - 2)$ .

**Exercice 2 : Plus petit cycle dans un graphe)** (5 points)

Soit un graphe  $G = (V, E)$  orienté quelconque sans pondération sur ses arcs. Dans cet exercice nous cherchons à trouver le plus petit cycle du graphe  $G$ .

**Question 2.1 :** Proposer une solution pour trouver le plus petit cycle dans le graphe donné à la figure 1: illustrer votre proposition à partir du graphe et en déroulant celle-ci.

**Question 2.2 :** Écrire sous forme algorithmique la fonction  $ppCycle(G[])$  qui prend en paramètre un graphe décrit sous la forme d'un tableau  $G$  de listes de voisins et retourne une liste  $C$  constituée des sommets qui composent le plus petit cycle.

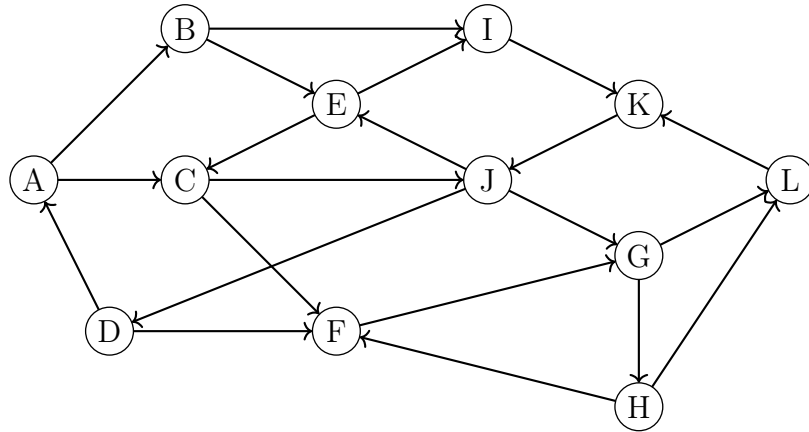


Figure 1: Graphe  $G = (V, E)$  orienté sans pondération sur ses arcs

**Question 2.3 :** *Cet algorithme est-il optimal? Prouver votre réponse.*

### **Correction Exercice 2 : Plus petit cycle dans un graphe orienté**

**Solution question 2.1 :** *Proposer une solution pour trouver le plus petit cycle dans le graphe donné à la figure 1: illustrer votre proposition à partir du graphe et déroulant celle-ci.*

Pour trouver les cycles une solution consiste à faire un parcours du graphe à partir des sommets. Un parcours en largeur est bien adapté, car celui-ci permet facilement de calculer la longueur du chemin qui relie le sommet initial au sommet courant, ce qui est nécessaire pour connaître la longueur d'un cycle. Un parcours en profondeur ne permettrait pas de trouver le plus petit cycle.

Nous partons d'un point quelconque et initions un parcours en largeur. Si nous rencontrons le sommet d'origine, alors c'est un cycle et le parcours peut être arrêté. Si nous ne rencontrons pas le sommet d'origine avant la fin du parcours, il n'y a pas de cycle incluant ce sommet. Comme dans les parcours en largeur classiques, il faut marquer les sommets une fois qu'ils sont atteints car, lorsque le parcours atteint un sommet déjà parcouru, cela signifie qu'un chemin plus court, ou de même longueur, a déjà été trouvé. Le cycle qui inclurait ce sommet est donc plus petit, ou égal, s'il passe par le premier chemin ayant permis d'atteindre le sommet.

Par exemple, si nous partons du sommet  $A$  de la figure 1, la première étape du parcours en largeur donne deux sommets:  $B$  et  $C$ . Aucune des arêtes ne revient au sommet  $A$ , ce qui est normal car il n'existe pas de cycle de longueur 1.

La seconde itération donne, depuis le sommet  $B$ , les sommets  $I$  et  $E$ , et depuis le sommet  $C$  les sommets  $J$  et  $F$ .

La troisième itération donne, à partir de  $I$ , le sommet  $K$ , à partir de  $E$  les sommets  $I$  et  $C$ , à partir de  $J$ , les sommets  $D$ ,  $G$  et  $E$ , et, à partir de  $F$ , le sommet  $G$ . A ce stade du parcours, nous pouvons constater qu'il existe un cycle avec les sommets  $E$ ,  $C$  et  $J$  mais le parcours en largeur réalisé ne permet pas de le mettre en évidence car les sommets

appartiennent à des branches différentes. Attention, ce n'est pas parce qu'un sommet est marqué comme déjà atteint qu'il y a un cycle. C'est le cas pour les sommets  $C$  et  $I$  que le parcours atteint une seconde fois depuis le sommet  $E$ .

La quatrième itération permet de revenir au sommet  $A$ , depuis le sommet  $D$ , et donc de trouver un cycle de longueur 4. La recherche peut s'arrêter là puisque, si nous trouvons un nouveau cycle, il sera soit de même taille soit plus long.

Nous reprenons la procédure pour chacun des sommets de  $G$  et nous notons la longueur du premier cycle trouvé. Le plus petit cycle est celui qui est le plus court parmi les cycles trouvés. Ici, le plus petit cycle trouvé est de longueur 3 avec le cycle  $\{C, J, E, C\}$ . À noter qu'il existe dans ce grand plusieurs cycle de longueur 3, qui peuvent donc tous prétendre être le plus petit cycle.

**Solution question 2.2 :** *Écrire sous forme algorithmique la fonction  $ppCycle(G[])$  qui prend en paramètre un graphe décrit sous la forme d'un tableau  $G[]$  de listes de voisins et retourne une liste  $C$  constituée des sommets qui composent le plus petit cycle.*

L'idée de base est donc de faire un parcours en largeur depuis chacun des sommets. L'algorithme 2 réalise la recherche du plus petit cycle du graphe  $G$  en appelant, pour chaque sommet, la fonction  $ppCycleSommet$  et en mémorisant la longueur et le cycle lorsque ceux-ci sont plus petit que ceux qui avaient été trouvés précédemment. Il termine en retournant les valeurs trouvées.

---

**Algorithme 2 :** Fonction trouvant le plus petit cycle dans le graphe  $G$ .

---

**Entrées :**  $G[]$ : tableau de listes de voisins

**Résultat :**  $Cmin$ : liste de sommets, **init** à  $\emptyset$

$lmin$ : entier, longueur du cycle, **init** à  $\infty$

```

1 Fonction  $ppCycle(G[])$ :
2 début
3   pour  $i \leftarrow 0$  à  $G.size()$  faire
4      $(C, l) = ppCycleSommet(G[], i)$ 
5     si  $(l > 0) \wedge (l < lmin)$  alors
6        $lmin \leftarrow l$ 
7        $Cmin \leftarrow C$ 
8   retourner  $Cmin, lmin$ 

```

---

La fonction  $ppCycleSommet$  est donnée par l'algorithme 3. Elle cherche le cycle le plus court, à partir d'un sommet  $s$  du graphe  $G$  donné dans un tableau de listes de voisins. Pour cela, elle utilise le parcours en largeur du cours, qui permet de calculer les distances et les filiations, puisque nous avons besoin de la distance pour connaître la taille du cycle. Nous y ajoutons l'identification du cycle le plus court (ligne 9) et la création de la liste  $C$  (lignes 10 à 17). Pour cela, nous remontons les filiations données par le tableau  $pere$  jusqu'au sommet initial  $s$ . Nous utilisons une pile sur laquelle chacun des sommets de la remontée est mis, avant d'en mettre le contenu dans la liste  $C$  (ligne 16).

**Solution question 2.3 :** *Est-ce que cet algorithme est optimal? Prouver votre réponse.*

---

**Algorithme 3** : Fonction trouvant le plus petit cycle dans le graphe  $G$  depuis un sommet  $s$ .

---

**Entrées** :  $G[]$ : tableau de listes de voisins

$s$ : sommet de départ

**Résultat** :  $C$ : liste de sommets, **init** à  $\emptyset$

$l$ : entier, longueur du cycle, **init** à 0

1 **Fonction**  $ppCycleSommet(G[], s)$  :

**Données** :  $f$ : file des sommets en cours **init** à  $\emptyset$

$couleur[u]$ :  $u \in \{BLANC, GRIS, NOIR\}$ , **initialisée** à  $BLANC$ ,  $\forall u \in V$

$p$ : pile

$d[]$ : tableau des distances à  $s$

$d[]$ : tableau des pères des sommets

2 **début**

3      $d[s] \leftarrow 0$

4      $f.add(s)$

5     **tant que**  $\neg f.empty()$  **faire**

6          $u \leftarrow f.poll()$

7         **pour**  $i \leftarrow 0$  à  $G[u].size() - 1$  **faire**

8              $v \leftarrow G[u].get(i)$

9             **si**  $v = s$  **alors**

10                  $l \leftarrow d[u] + 1$

11                  $p.push(v)$

12                 **tant que**  $u \neq s$  **faire**

13                      $p.push(u)$

14                      $u \leftarrow pere[u]$

15                  $p.push(s)$

16                 **tant que**  $\neg p.empty()$  **faire**  $C \leftarrow p.pop()$

17                 **retourner**  $C, l$

18             **si**  $couleur[v] = BLANC$  **alors**

19                  $couleur[v] \leftarrow GRIS$

20                  $d[v] \leftarrow d[u] + 1$

21                  $pere[v] \leftarrow u$

22                  $f.add(v)$

23     **retourner**  $C, l$

---

Je dois reconnaître que cette question était mal posée. Ce qui était attendu était plutôt la réponse à “est-ce que cet algorithme donne une solution optimale”. C’est à cette question que répond la suite.

Le parcours en largeur permet de trouver le chemin le plus court entre deux sommets. Ainsi le cycle trouvé en un sommet est-il le cycle le plus court incluant ce sommet. Puisque, pour chaque sommet, nous avons la longueur du cycle le plus court qui inclut ce sommet alors en prenant le minimum de chacun de ces cycles, nous obtenons bien le cycle le plus court du graphe.



### Exercice 3 : Liste de commandes (5 points)

Une entreprise doit réaliser une liste de commandes  $c_i$ . Chacune de ces commandes prend le même temps à être réalisée et l'entreprise ne peut réaliser qu'une seule commande à la fois. Les dates  $d_i$  auxquelles les commandes doivent être finies au plus tard ne sont pas les mêmes et les contrats des commandes prévoient qu'une pénalité  $p_i$  est à payer en cas de retard sur la réalisation de la commande. Comme le gain par commande est le même, l'entreprise cherche à minimiser les pénalités qu'elle aura à payer si elle ne peut pas finir certaines commandes à temps.

|       |   |   |   |   |   |   |   |   |
|-------|---|---|---|---|---|---|---|---|
| $c_i$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| $d_i$ | 2 | 2 | 2 | 3 | 4 | 5 | 6 | 6 |
| $p_i$ | 4 | 3 | 1 | 3 | 2 | 3 | 2 | 1 |

Table 1: Liste de commandes

**Question 3.1 :** Donner un ordonnancement des commandes données dans le tableau 1 qui minimise les pénalités de retard.

À noter que, puisque toutes les commandes ont la même durée, le problème est équivalent au cas où toutes les commandes ont une durée de 1. Les commandes données dans le tableau ont donc une durée unitaire.

**Question 3.2 :** Proposer une solution gloutonne pour permettre de choisir la liste de commandes  $CR$  à réaliser, parmi un tableau de  $n$  commandes  $C[]$ , pour minimiser les pénalités.

**Question 3.3 :** Donner l'algorithme qui implémente votre solution

Pour obtenir la date limite de la commande  $i$  vous pouvez utiliser la notation  $C[i].d$ , de la même manière vous pouvez utiliser la notation  $C[i].p$  pour obtenir la pénalité associée. De plus, vous pouvez supposer l'existence de deux fonctions:  $comParD(C, L)$  et  $comParP(C, L)$  qui rendent une liste triée des numéros des commandes contenus dans la liste  $L$ , respectivement en fonction des dates limites et des pénalités.

**Question 3.4 :** Votre solution est-elle optimale? Prouvez votre réponse.

### Correction Exercice 3 : Ordonnancement

**Solution question 3.1 :** Donner un ordonnancement des commandes données dans le tableau 1 qui minimise les pénalités de retard.

L'ordonnancement qui minimise les pénalités est composé des tâches  $C_1, C_2, C_4, C_5, C_6$  et  $C_7$ . La pénalité associée est de 2. Il est facile de voir que la tâche  $C_3$  ne peut pas faire partie des commandes à traiter car il y a 3 tâches à faire avant la date 2:  $C_1, C_2, C_3$ . Les tâches  $C_1$  et  $C_2$  ont les plus fortes pénalité, il faut donc plutôt faire celles-ci en priorité. De même pour les tâches  $C_7$  et  $C_8$ , qui sont en conflit pour la date 6 et c'est  $C_8$  qui ne sera pas faite car elle a une pénalité plus faible

**Solution question 3.2 :** Proposer une solution gloutonne pour permettre de choisir la liste de commandes  $CR$  à réaliser, parmi un tableau de  $n$  commandes  $C[]$ , pour minimiser

les pénalités.

Pour minimiser la pénalité il faut choisir les commandes qui doivent être faites avant leur date au plus tard, celles qui n'engendreront pas de pénalité. Les autres commandes pourront être faites après, à n'importe quelle date, puisque l'entreprise payera la pénalité. Nous cherchons donc une liste de commandes compatibles qui peuvent être réalisées avant leur date au plus tard.

D'après ce que nous avons observé à la question précédente, il faut commencer par faire les commandes ayant les pénalités les plus fortes. L'algorithme proposé est donc de trier les commandes par ordre décroissant de pénalité. Nous itérons ensuite sur cette liste triée et chaque fois qu'une commande peut être réalisée avec la liste déjà existante, elle est ajoutée à la liste. Sinon, elle ne fera pas partie de la liste finale.

### **Solution question 3.3 :** Donner l'algorithme qui implémente votre solution

L'algorithme 5 permet d'obtenir la liste des commandes à réaliser  $CR$ , initialement vide. Il crée une liste  $CPP$  des numéros de commande qu'il trie en fonction de la pénalité avec la fonction  $comParP$  (lignes 3 et 4). Ensuite, pour chacune des commandes de cette liste triée, il l'ajoute à la liste  $CR$  et vérifie si l'ordonnancement est réalisable à l'aide de la fonction  $testOrdo$ , donnée par l'algorithme 5 (lignes 5 à 8). Si la commande n'est pas compatible avec les contraintes de dates des commandes déjà sélectionnées, elle est retirée de la liste (ligne 9). Lorsque toutes les commandes ont été testées la liste  $CR$  est retournée.

Pour tester la faisabilité d'un ordonnancement pour les commandes d'une liste il est possible de les trier par date au plus tard avec la fonction  $comParD$ . Les tâches ayant les dates au plus tard les plus petites sont faites en premier. Pour chaque commande la fonction vérifie qu'elle termine avant sa date au plus tard. À noter que le test doit se faire par rapport à la fin de la commande et non pas par rapport au début, d'où la valeur de  $i + 1$ . Si une commande ne peut pas terminer avant sa date au plus tard la fonction retourne *faux*, si toutes les commandes respectent leur date au plus tôt la fonction retourne *vrai*.

---

#### **Algorithme 4 :** Fonction de vérification de validé d'un ordonnancement

---

**Entrées :**  $C[]$ : tableaux des commandes

$L$ : liste des commandes

**Résultat :** booléen indiquant si un ordonnancement existe

```
1 Fonction testOrdo(C,L)
2 début
3    $L \leftarrow comParD(C, L)$ 
4   pour  $i \leftarrow 0$  à  $L.size() - 1$  faire
5      $c \leftarrow L.get(i)$ 
6     if  $C[c].d < i + 1$  then retourner faux
7   retourner vrai
```

---

---

**Algorithme 5** : Algorithme de calcul de la liste des commandes à réaliser.

---

**Entrées** :  $C[]$ : tableaux des commandes

**Résultat** :  $CR$ : liste des commandes à réaliser, **init** à  $\emptyset$

1 Fonction **ordoCommandes**( $C$ )

**Données** :

$L$ : liste de commandes, **init** à  $\emptyset$

2 **début**

```
3   pour  $i \leftarrow 0$  à  $C.size() - 1$  faire  $L.add(i)$ 
4    $L \leftarrow comParP(C, CPP)$ 
5   pour  $i \leftarrow 0$  à  $L.size() - 1$  faire
6        $com \leftarrow L.get(i)$ 
7        $CR.add(com)$ 
8        $ordo \leftarrow testOrdo(C, CR)$ 
9       si  $ordo \neq true$  alors  $CR.remove(com)$ 
10  retourner  $CR$ 
```

---

**Solution question 3.4** : *Est-elle optimale? Prouvez votre réponse.*

La solution proposée est optimale.

On peut le prouver par l'absurde en supposant qu'une commande ne fait pas partie de la liste obtenue mais permet de diminuer la pénalité. Puisque cette commande ne fait pas partie de la liste, c'est qu'il n'y avait pas de place pour la mettre avant sa date limite. Cela implique que, pour mettre cette commande dans la liste, il faut en enlever une qui a été choisie. Or la commande qui est dans la liste a forcément une pénalité inférieure puisque l'algorithme classe les commandes par ordre de pénalité. Donc, en échangeant les deux commandes, on augmente la pénalité et la solution obtenue a une moins bonne pénalité globale, ce qui est contraire à l'hypothèse de départ. Donc la solution obtenue est bien optimale.